

Efficient and Flexible Low-Power NTT for Lattice-Based Cryptography

Presented by

Krishna Gorai(T25111), Achal Shah(T25119), Ayan Garg(B23484)



Submitted To

Dr. Srinivasu Bodapati (Associate Professor)
School of Computing and Electrical Engineering
Indian Institute of Technology Mandi

Problem Statement

- Classical cryptographic systems such as RSA and ECC are vulnerable to attacks from quantum computers.
- Lattice-based cryptography has emerged as a promising post-quantum solution, but it relies heavily on polynomial multiplication.
- Polynomial multiplication using conventional methods is computationally expensive and inefficient for hardware implementation.
- The Number Theoretic Transform (NTT) enables faster computation, but existing implementations face challenges in power consumption and hardware complexity.

Objective:

- Design an efficient and low-power hardware architecture for NTT/INTT operations suitable for lattice-based cryptographic systems.

NTT: Number Theoretic Transform

What is NTT

A variant of FFT operating over a finite field (integers mod q) instead of complex numbers

$$C = \text{NTT}^{-1}[\text{NTT}(a) \circ \text{NTT}(b)]$$

Where $\circ$ = point wise multiplication

- Only Integer Arithmetic is Possible
- Needs $\log(n)$ precomputed Twiddle Factors

Native Multiplication: Too slow for large n , hardware dividers needed $O(n^2)$

NTT Multiplication: $n \cdot \log(n)$ clock cycles, single-port RAM $O(n \log n)$

NTT hardware advantages:

1. Integer-only math
2. Small LUT ($\log n$)

Algorithm: NTT Transform

1. Bit Reversal: Reorder input array using bit-reversal

2. Butterfly Computation

For $m = 2$ to n (doubling each stage):

- Compute twiddle factors (w_m, w)

For each block:

- $t = a[k + j + m/2] \times w$
- $u = a[k + j]$
- $a[k + j] = u + t$
- $a[k + j + m/2] = u - t$
- Update: $w = w \times w_m$

3. Final Stage: Same butterfly computation for $m = n$

Core Idea:

- Divide-and-conquer using butterfly operations
- Works similar to FFT but in modular arithmetic

Key Features:

- Time Complexity: $O(n \log n)$
- Operates modulo q (finite field)
- Uses precomputed twiddle factors

Inverse NTT:

- Uses inverse twiddle factors
- Recovers original input

Algorithm: Barrett Modulo Reduction

Input: Integer a , constants m and k

Output: $a \bmod q$

Steps:

$$1. u = (a \times m) \gg k$$

$$2. u = u \times q$$

$$3. a = a - u$$

$$4. \text{If } a \geq q \rightarrow a = a - q$$

Key Idea:

Replaces division with multiplication + shift

Efficient for hardware (used in NTT, cryptography)

Algorithm: Montgomery Modulo Reduction

Input: Integer a (Montgomery form), constants R , $\log(R)$, q'

Output: $a \bmod q$

Steps:

1. $u = a \times q'$
2. $u = u \text{ AND } (R - 1)$
3. $a = a + (u \times q)$
4. $a = a \gg \log(R)$
5. If $a \geq q \rightarrow a = a - q$

Key Idea:

Replaces division with multiplication, masking, and shift

Efficient for hardware and large integer arithmetic (used in cryptography, NTT)

Algorithm: Optimized NTT Transform(First Round)

1. Bit Reversal: Reorder input array using bit-reversal
2. Butterfly Computation For $m = 2$ to $n/2$ (doubling each stage):

Load ω_m from Montgomery LUT

$\omega = \text{Montgomery LUT}(\text{index})$ or $R \bmod q$ for NTT⁻¹

For each block:

- $t1 = \text{mont}(a[k+j+m/2] \times \omega)$
- $t2 = \text{mont}(a[k+m+j+m/2] \times \omega)$
- $a[k+j] = \text{barrett}(u1 + t1)$
- $a[k+j+m/2] = \text{barrett}(u1 - t1 + q)$
- $a[k+m+j] = \text{barrett}(u2 + t2)$
- $a[k+m+j+m/2] = \text{barrett}(u2 - t2 + q)$
- Update: $\omega = \text{mont}(\omega \times \omega_m)$

3. Final Stage: Results written back to MEM(k+j) and MEM(k+j+m/2)

Core Idea:

- Same butterfly structure as Algorithm 1 but with modular reduction integrated
- Replaces direct multiplication with Montgomery reduction

Key Features:

- Montgomery reduction for modulo multiplication
- Barrett reduction for modulo addition/subtraction
- Twiddle factors stored in Montgomery domain

Why it's better:

- No hardware dividers needed
- ω computed at runtime — no large LUT
- q added during subtraction to prevent negative inputs

Algorithm: Optimized NTT Transform (Last Round)

1. Initialization

- $m = n$
- Load ω_m and ω from Montgomery LUT
- Set $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ from pre-calculated values

2. Butterfly Computation For $j = 0$ to $m/2 - 1$ (step size 2):

- $t1 = \text{mont}(t1 \times \omega)$
- Update: $\omega = \text{mont}(\omega \times \omega_m)$
- $t2 = \text{mont}(t2 \times \omega)$
- $a[j] = \text{barrett}(u1 + t1)$
- $a[j+m/2] = \text{barrett}(u1 - t1 + q)$
- $a[j+1] = \text{barrett}(u2 + t2)$
- $a[j+m/2+1] = \text{barrett}(u2 - t2 + q)$

3. If Inverse NTT — Post-processing:

- $a[j] = \text{mont}(\alpha_1 \times a[j])$
- $a[j+m/2] = \text{mont}(\alpha_3 \times a[j+m/2])$
- $a[j+1] = \text{mont}(\alpha_2 \times a[j+1])$
- $a[j+m/2+1] = \text{mont}(\alpha_4 \times a[j+m/2+1])$
- Update $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ using $\beta = \omega_n^{-1} \cdot R$

4. Write results back to MEMj and MEMj+1

Core Idea:

- Handles the final NTT round separately for efficiency
- Integrates NTT⁻¹ post-processing into the last round itself

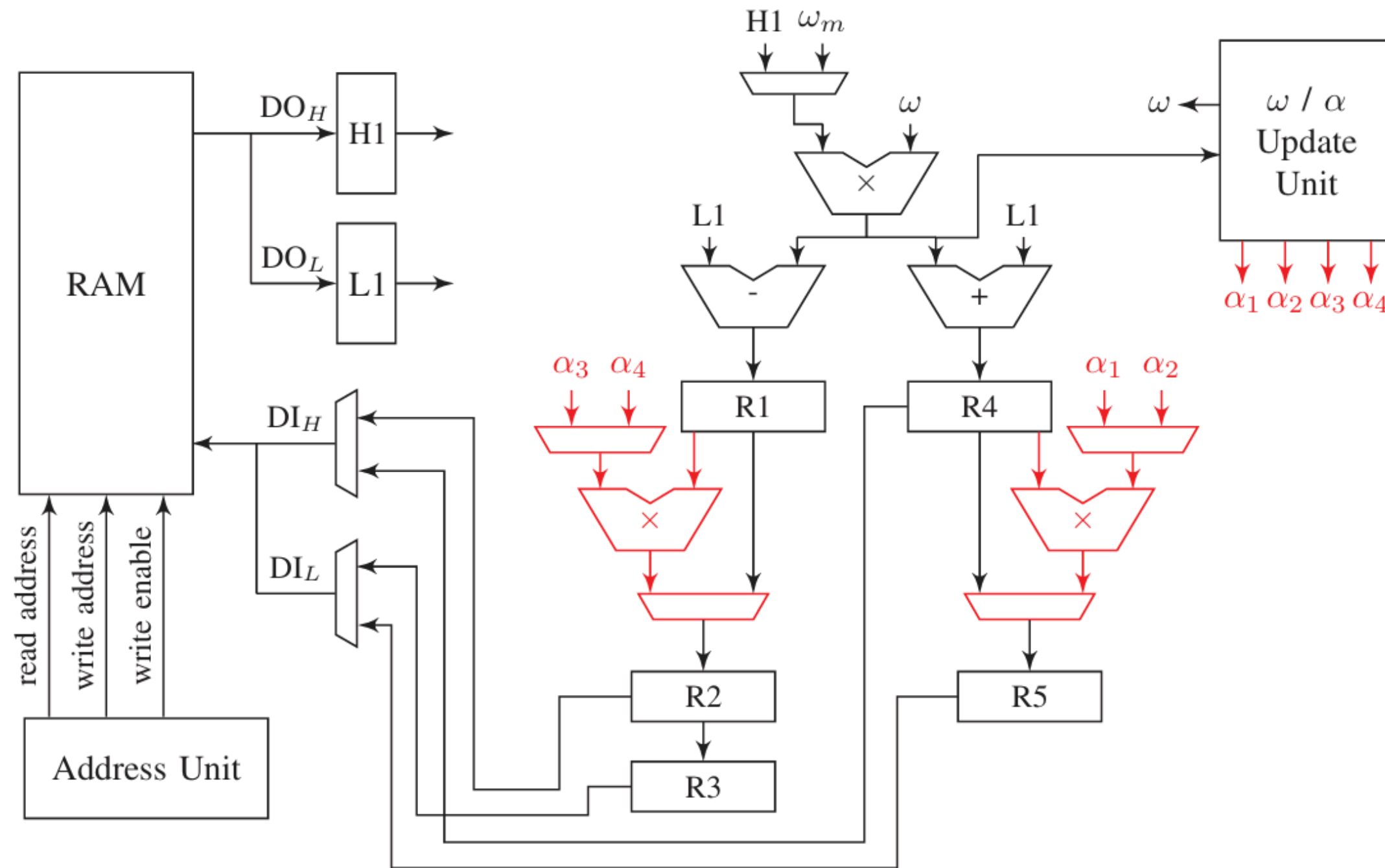
Key Features:

- Time Complexity: $O(n \log n)$
- Processes 2 memory locations per iteration
- Saves $n/4$ clock cycles vs. standard last round

Inverse NTT:

- $\alpha_1 = n^{-1} \cdot R$
- $\alpha_2 = n^{-1} \cdot \gamma_n^{-1} \cdot R$
- $\alpha_3 = n^{-1} \cdot \gamma_n^{-n/2} \cdot R$
- $\alpha_4 = n^{-1} \cdot \gamma_n^{-n/2-1} \cdot R$
- **Eliminates separate post-processing₈ pass — saves at least $2n$ clock cycles**

Architecture of optimised NTT/INTT Transform



Four Main Modules:

RAM: Stores two 16-bit coefficients per 32-bit memory line.

Address Unit: Controls read/write addresses for RAM access.

ω/α Update Unit — Updates twiddle factor ω and scaling factors α_1 – α_4 each cycle.

Core Operation: Butterfly unit performing Montgomery multiplication and Barrett reduction.

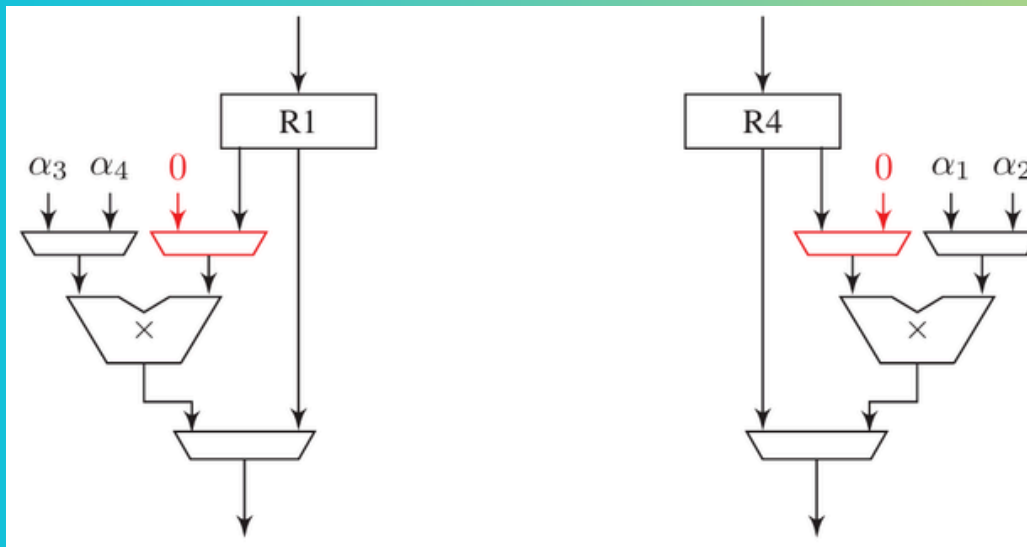
Registers:

H1, L1: Hold high and low 16-bit coefficients from RAM.

R1–R5: Store intermediate results and enable coefficient swapping.

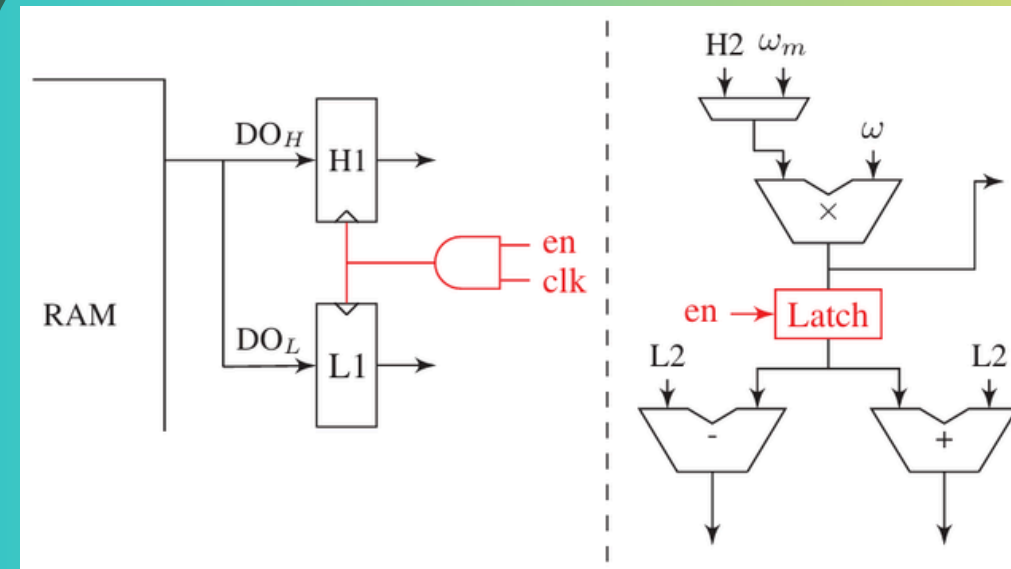
Red components — Additional multipliers active only during NTT⁻¹.

Power Optimisation Techniques



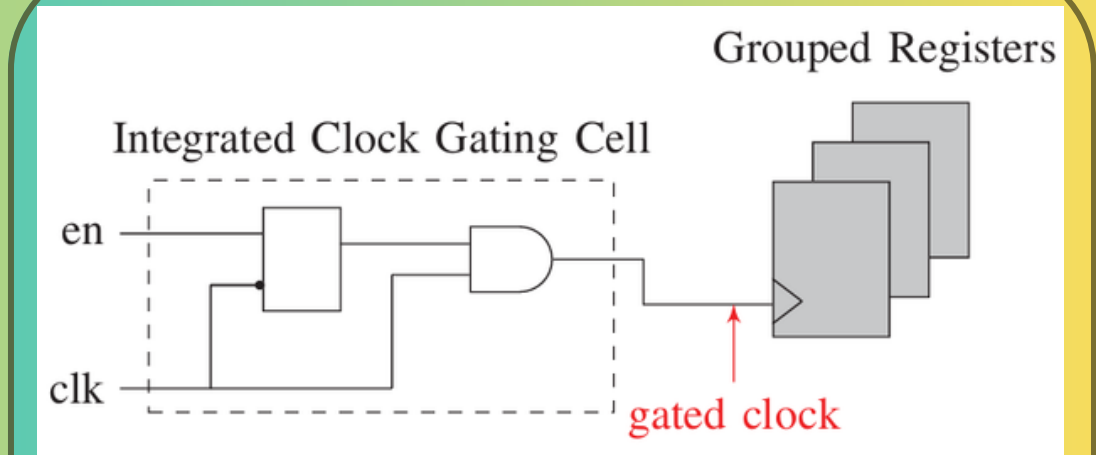
Operand Isolation: NTT^{-1} Multipliers

Inputs to a multipliers are forced to zero outside the last round, preventing unnecessary switching and reducing dynamic power.



Operand Isolation: Input Registers

H1 and L1 are clock-gated during write cycles. A latch after the first multiplier blocks results from propagating when not needed.



Clock Gating: Integrated Clock Gating Cell

Registers with a common enable signal are grouped. A gated clock disables inactive registers, reducing dynamic power to zero during idle cycles.

FPGA Results - Zynq-7020 xc7z020clg484-1

Resource	Used	Avail.	Util%	Paper[1]
LUT as Logic	953	53,200	1.79	—
LUT as Dist.RAM	268	17,400	1.54	—
LUT total	1,221	53,200	2.30%	980
Registers (FF)	573	106,400	0.54	395
DSP48E1	25	220	11.36%	26
CARRY4	81	13,300	0.61	—

ASIC Results - UMC 130nm

TT corner: 1.2V, 25°C (synthesis) · SS corner: 1.08V, 125°C (P&R sign-off)

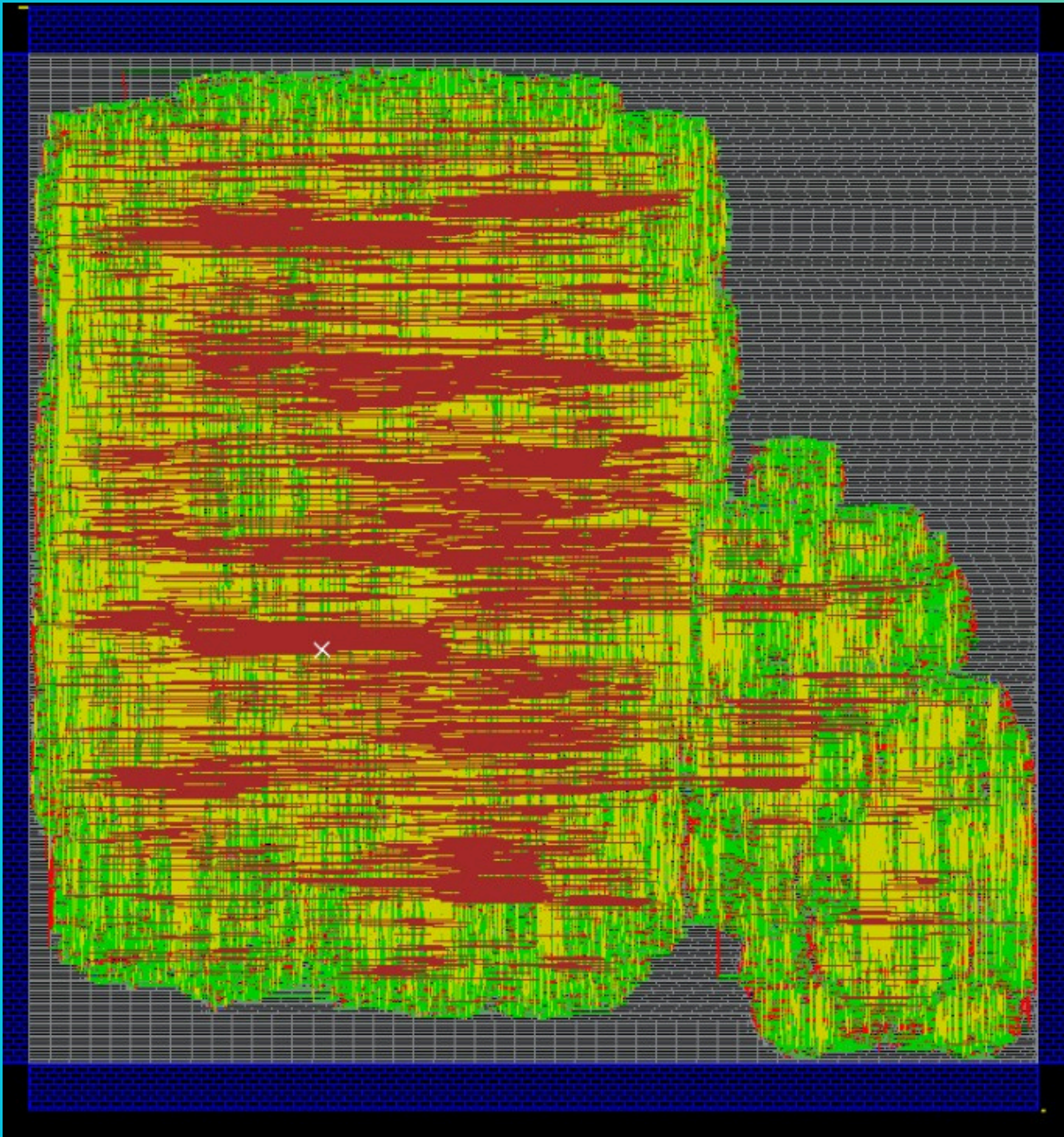


Fig : ASIC Implementation on UMC 130nm

Synthesis(Genus)

Metric	Value
Total cells	45,120
RAM (FF)	33,500
Logic	11,620
Total area	854,844 μm^2
Logic area	178,473 μm^2
WNS (TT)	+472 ps

P&R Summary

Die size	1700 × 1700 μm
Core size	1276 × 1276 μm
Utilisation	~52%
Metal layers	5 (M1–M5)
DRC	0 violations ✓
GDS	Exported ✓

Timing Closure – 25 MHz

Stage	WNS	Status
Post-synth	+472 ps	MET ✓
Pre-CTS	-13.1 ns	Expected
Post-CTS	+6 ps	MET ✓

Post-Route Power(SS)

Component	mW	%
Sequential	8.312	47.1
Combinational	7.434	42.1
Clock tree	1.908	10.8
Internal	10.802	61.2
Switching	6.054	34.3
Leakage	0.797	4.5
TOTAL	17.653	100

Comparison with Reference – Fritzmann & Sepulveda, IEEE HOST 2019

Metric	Reference [1] (HOST 2019)	This Work	Remark
FPGA device	Zynq-7000 (Zedboard)	Zynq-7020 xc7z020	Same family ✓
FPGA LUTs	980	1,221	~25% higher (configurability)
FPGA DSPs	26	25	≈ identical ✓
ASIC technology	UMC 65 nm	UMC 130 nm	Different node
ASIC frequency	25 MHz	25 MHz	Matched ✓
Post-route WNS	Met	+6 ps (SS corner)	Met ✓
Total cells	14,352	45,120	3.1× due to FF RAM
Total area (μm ²)	329,598	854,844	2.6× (node + RAM)
Logic area	~300,000 μm ²	178,473 μm ²	Logic smaller ✓
RAM type	SRAM macro	FF-based (PDK limit)	No macro in PDK
P_static	19.2 μW	0.797 mW	130nm leakage
P_dynamic	2.920 mW	16.856 mW	5.8× (node + FF RAM)
P_total	2.940 mW	17.653 mW	Expected for node

Conclusion

What Was Achieved

- **RTL + Algorithms** : All algorithms of [1] in synthesisable Verilog-2005: Montgomery (Alg 3), Barrett (Alg 2), NTT (Alg 4), INTT with folded post-processing (Alg 5), clock gating, operand isolation.
- **Verification** : Python model + Vivado behavioural simulation PASSED. Forward NTT and $\text{INTT}(\text{NTT}(a))=a$ round-trip both confirmed.
- **FPGA** : Zynq-7020: 1,221 LUTs, 25 DSPs — matching [1].
- **ASIC** : UMC 130nm: 25 MHz timing closure (WNS = +6 ps at SS worst-case). Post-route power 17.65 mW.

75 % of the reference paper [1] was implemented , except for side channel security.

References

- [1] T. Fritzmann and J. Sepulveda, "Efficient and Flexible Low-Power NTT for Lattice-Based Cryptography," in Proc. IEEE HOST, McLean, VA, USA, 2019, pp. 141–150.**
- [2] P. L. Montgomery, "Modular Multiplication Without Trial Division," Mathematics of Computation, vol. 44, no. 170, pp. 519–521, 1985.**
- [3] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965.**
- [4] R. Avanzi et al., "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation," NIST PQC Round 3 Submission, 2021.**

Thank You